LPC2148 - GPIO



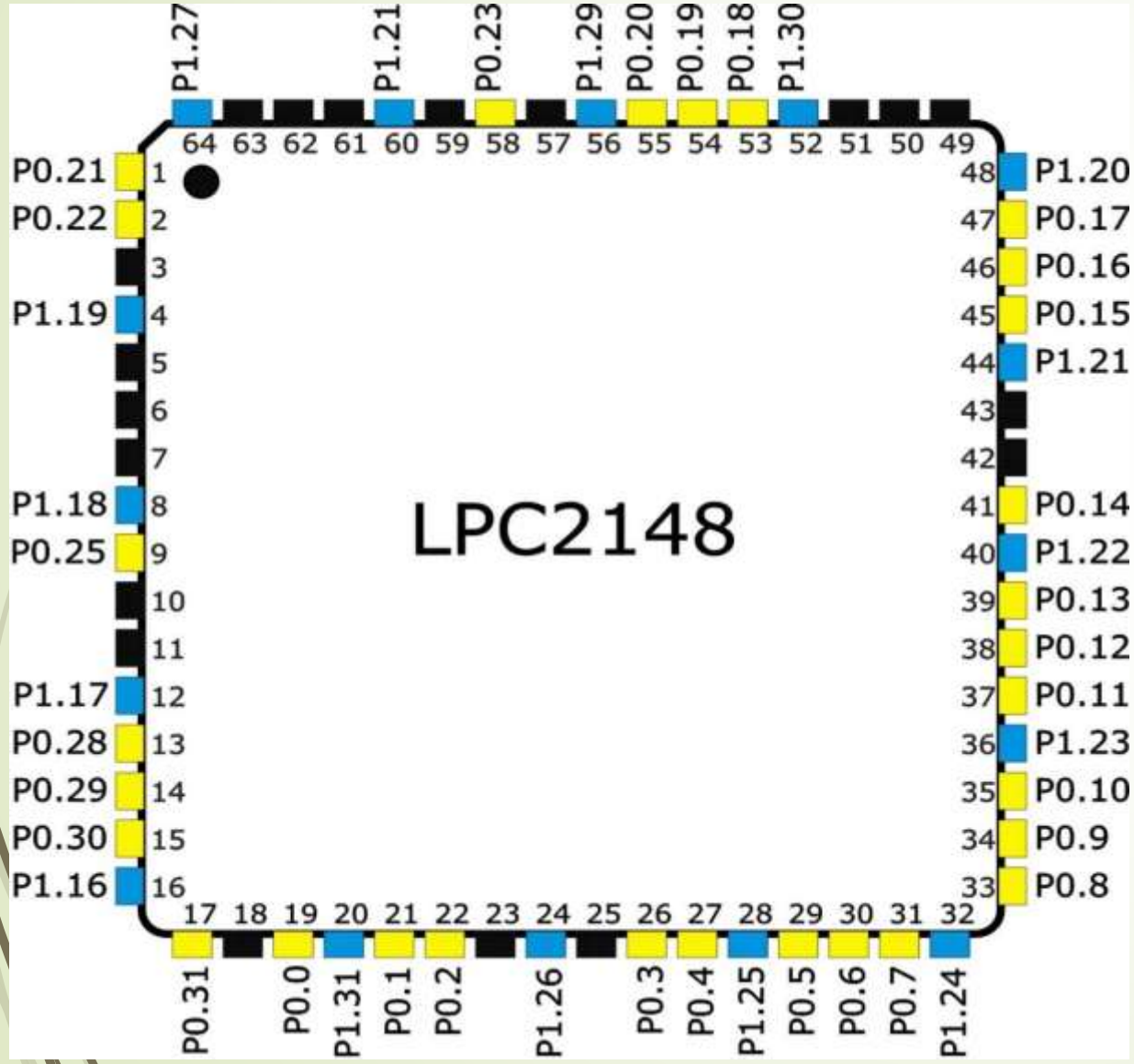
Parallel Port –Port 0

- Port 0 = 32 Bits ⟶ **(P0.0 to P0.31)**
- P0.24, P0.26 and P0.27 are not available
- So (32-3) = 29 Pins are Available
- Out of 29 Pins, 28 Pins ⟶ Bidirectional, Pin 31 ⟶ O/P Only

Parallel Port –Port 1

- Port 1 = 16 Bits → P1.16 to P1.31
- P1.0 to P1.15 of port 1 are
- P0 = 29 (28 Bidirectional + 1 output only)
- P1 = 16 (16 Bidirectional)
- Total = 45 GPIOs

Parallel Port

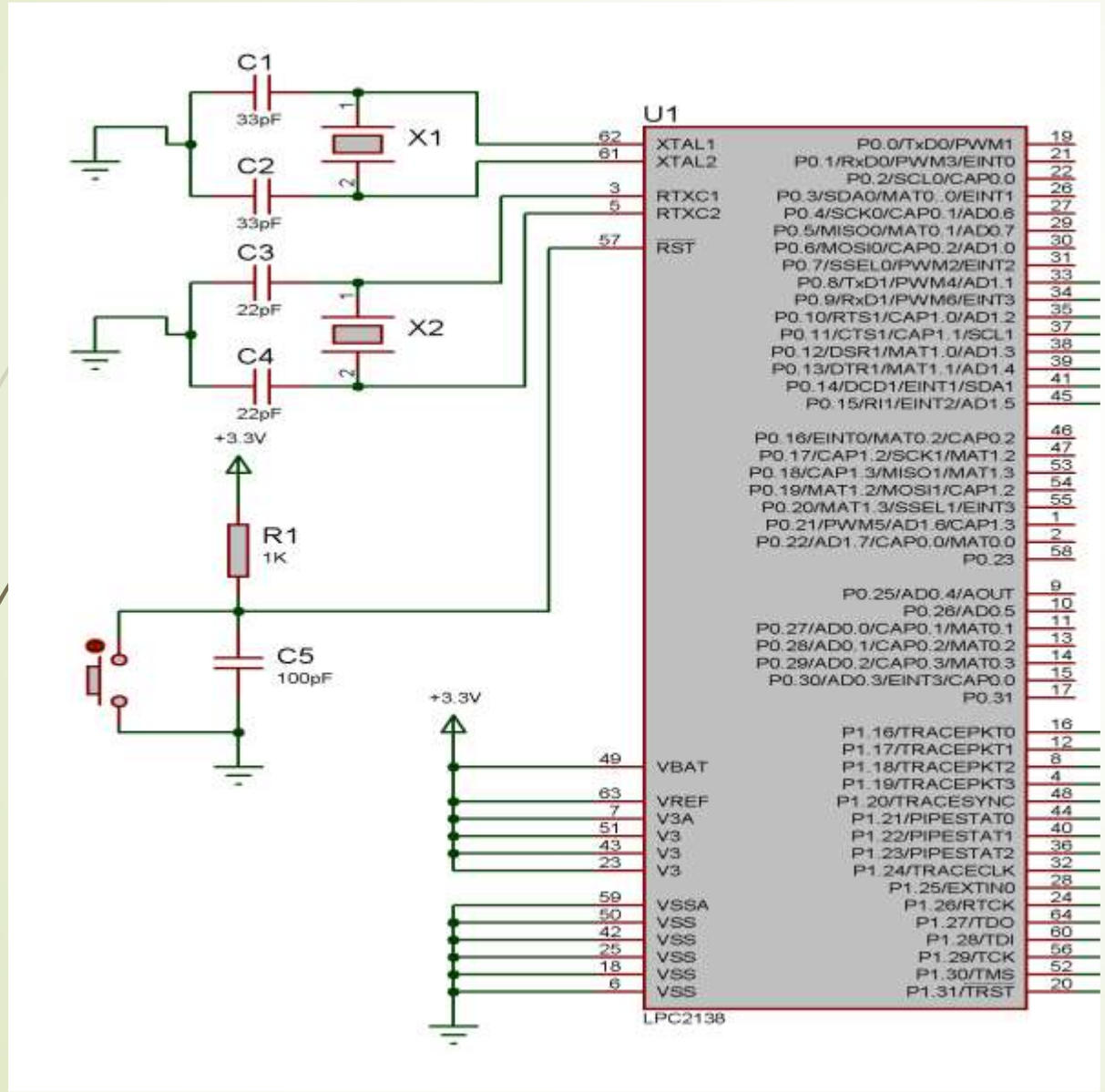


P0 Pins = 29 Pins

P1 Pins = 16 pins

Total = 45 Pins

Pins Used For GPIO in our Board



P0 – (P0.8-P0.15)

P1 – (P1.16-P1.23)

P2 – (P1.24-P1.31)

SFRs Used

P0

- IODIR0 → to set the direction
- IOSET0 → to set a Pin
- IOCLR0 → to clear a Pin
- IOPIN0 → to Read a Pin

P1

- IODIR1 → to set the direction
- IOSET1 → to set a Pin
- IOCLR1 → to clear a Pin
- IOPIN1 → to Read a Pin

IODIR

1 → Output **0 → Input**

- IODIR controls **the 'direction'** of the GPIO pin.
- You use this register to set a GPIO pin to either Input (0) or Output (1).
- To Configure **GPIO 0.10** to 'Output' (1).

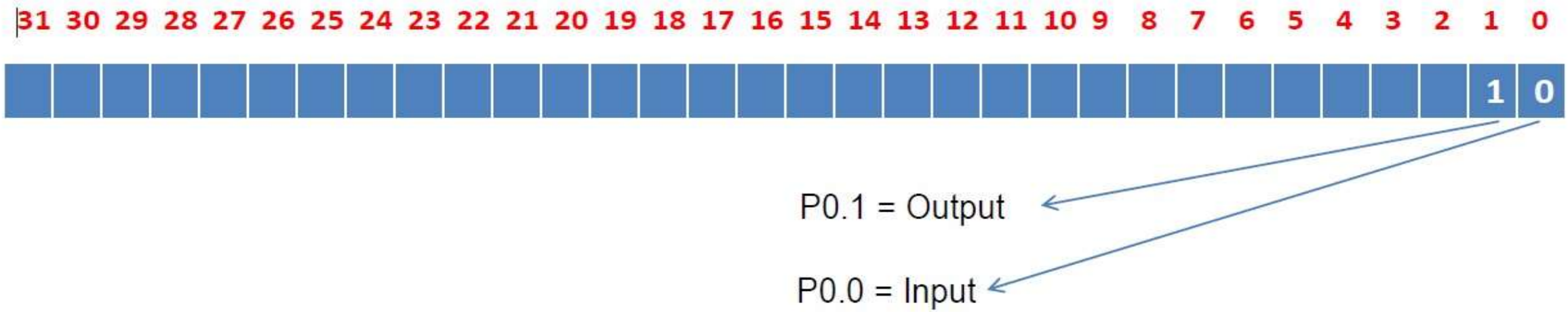
```
GPIO0_IODIR |= (1 << 10);
```

- To Configure **GPIO 0.10**to 'Input' (0).

```
GPIO0_IODIR &= ~(1 << 10);
```

IODIR0

1 ➔ Output
0 ➔ Input



Writing a 1 ➔ Pin is configured as Output

Writing a 0 ➔ Pin is configured as Input

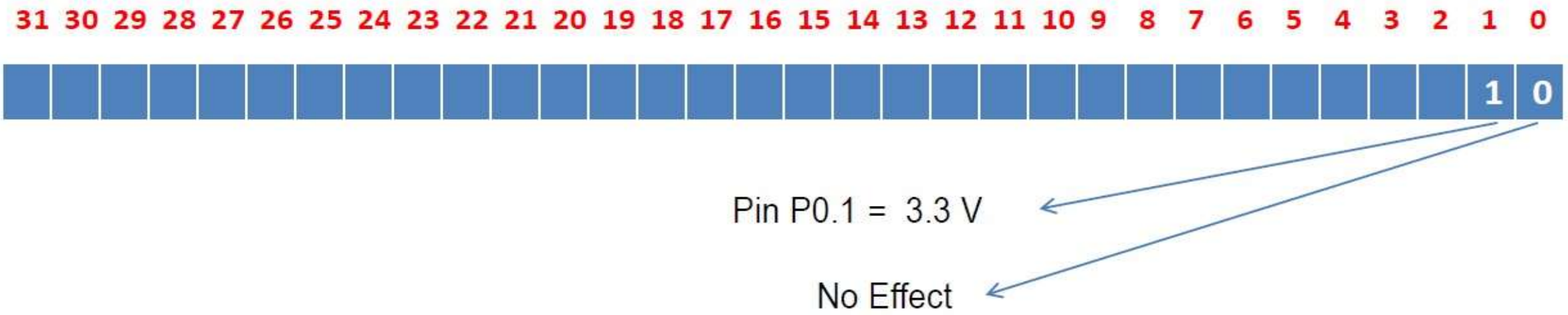
IOSET and IOCLR

- If GPIO pin is set as *Output*(using the IODIR register), then

- **IOSET** is used to set GPIO pin to **'high'**(providing a small 3.3V electrical output)

- **IOCLR** is used to clear it to **'low'**(providing a connection to GND).

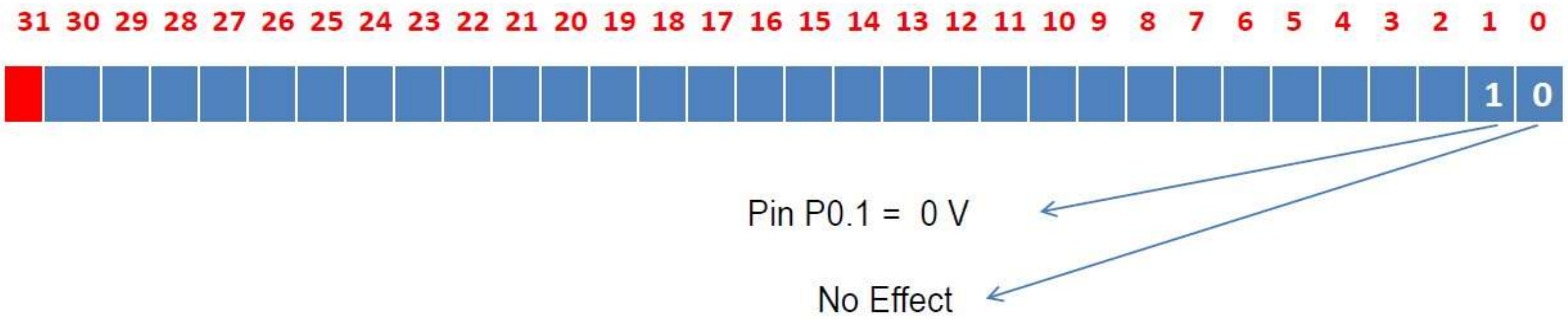
IOSET0



Writing a 1 ➔ Pin is set to **High**

Writing a 0 ➔ **No Effect**

IOCLR0



Writing a 1 ➔ Pin is cleared to **Low**

Writing a 0 ➔ No Effect

A small Example

- // Make sure GPIO 0.10 and 0.11 are set to output
- IODIR0 |= (1 << 10) | (1 << 11);
- // Turn the LEDs **off** using IOCLR (which gives a GND connection)
- IOCLR0 |= (1 << 10) | (1 << 11);
- // Turn the LEDs **on** using IOSET (which supplies 3.3V
- IOSET0 |= (1 << 10) | (1 << 11);

IOPIN

- Regardless of whether GPIO pin's direction is set to Input or Output,

- **IO PIN** register is used to **read the current 'state'** of every GPIO pin(all 32 pins in GPIO).

- A 1 value means that the pin is currently 'high', and a 0 value means the pin is currently 'low'.

A Small Example

```
int get Pin State(int pin Number)

{

// Read the current state of all pins in GPIO block 0

int pin Block State = GPIO0_IOPIN;

// Read the value of 'pin Number'

int pin State = (pin Block State & (1 << pin Number)) ? 1 : 0;

// Return the value of pin State

return pin State;

}
```